第5章 第10节 RAG检索优化：让答案更准确

- 来源: https://class.imooc.com/lesson/5273#mid=104922
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: 第一个RAG应用：游戏知识问答系统
- 课节: 图文 1-4 RAG检索优化：让答案更准确

1-4 RAG检索优化：让答案更准确

第5章 第10节 RAG检索优化：让答案更准确

> [!TIP] 阅读指南 在前面几节，我们已经走完了RAG的大部分流程： 用户提问 → 向量检索 → 【上下文拼接】 → 大模型生成 → 返回答案 1 代码块 上下文优化，就发生在"向量检索"和"大模型生成"之间这个关键环节。 你可能会问：检索不是已经完成了吗？我们已经拿到了最相关的3-5个chunk，接下来直接丢给大模型生成答案不就行了？ 没错，理论上是这样。但实际上，这个"拼接上下文"的环节，藏着巨大的优化空间： 给AI多少chunk？ 3个还是10个？ 每个chunk要完整给吗？ 还是可以压缩？ 怎么组织这些chunk？ 直接拼接还是加点结构？ 如何控制成本？ 每次对话都传几千字，钱包受得了吗？ RAG的上下文管理，本质上就是在回答一个问题： 给AI多少资料才够？太少了答不准，太多了成本飙升、效果还会变差。 这节课，我就来聊聊这门"给多少、怎么给"的艺术。

10.1 上下文的困境：多了不行，少了也不行

一个例子

某电商公司做了个AI客服，把商品FAQ全塞进上下文：

- 知识库：500个常见问题，总计15万字
- 用户问： “这款手机支持5G吗？”
- 系统检索：返回Top-10相关FAQ，共1.2万字

他们的想法是： “既然能检索到10个相关FAQ，那就都给AI吧，信息越多越好！”

运行一周后，问题来了：

- 成本失控
- 每次对话输入：1.2万字（问题100字 + 上下文1.2万字）
- 按通义千问价格：输入0.0008元/千tokens，每次对话成本约0.12元
- 日均1000次咨询，月成本：3600元
- 而正确的上下文只需要3个FAQ（3000字），成本可降至900元/月
- 答案质量下降

```
用户："这款手机支持5G吗？"

AI回答："根据资料，这款手机的网络支持情况如下：
1. 支持2G/3G/4G网络
2. 支持Wi-Fi 6
3. 支持蓝牙5.2
4. 关于5G，部分型号支持，部分型号不支持
5. 具体请查看产品型号说明..."
```

那这有什么问题呢？上下文包含了太多不同型号的信息，AI被10个FAQ搞晕了，不知道用户问的是哪个型号。原本一句话能回答的问题，变成了模棱两可的长篇大论

- 响应速度变慢

原本1秒返回答案，现在需要更久（模型要处理更多token）。

这个案例让我们明白：上下文不是越多越好。

另一个极端：吝啬的上下文。

还是这家电商公司，吸取教训后，走向了另一个极端：只返回Top-1最相关的FAQ，每次上下文只有300字，成本降到了最低。

结果又出问题了：

```
用户："这款手机拍照怎么样？"

检索到的Top-1：
"该手机采用6400万像素主摄，支持OIS光学防抖。"

AI回答："该手机采用6400万像素主摄，支持OIS光学防抖。"
```

用户追问："夜景模式呢？"

检索到的Top-1（换了）： “夜景模式支持AI降噪，最长曝光8秒。”

AI回答： “夜景模式支持AI降噪，最长曝光8秒。”

那这有什么问题呢？每个答案都是正确的，但都太片面，像个复读机，用户得问好几次才能了解全貌。

这个案例也告诉我们：上下文也不能太少。

上下文的黄金平衡点

那到底多少才合适？

没有标准答案。但有判断标准：

- 能回答问题的最小信息集

不是越多越好，而是刚好够用——就像给AI一张简明扼要的"小抄"。

- 具体数字参考
- 简单问答：1-2个chunk（500-1000字）
- 中等问题：3-5个chunk（1500-3000字）
- 复杂问题：5-10个chunk（3000-6000字）
- 动态调整

根据问题复杂度、检索相似度和实际效果动态调整上下文数量。

10.2 上下文优化的三个方面

角度1：Top-K参数调优

Top-K 就是从向量库中检索出"最相关的K个chunk"。

常见误区：很多人觉得K越大越好， “宁可多给，不能少给” 。

```
# 检索Top-K
results = collection.query(
    query_texts=["这款手机支持5G吗"],
    n_results=10  # K=10，真的需要这么多吗？
)
```

实测对比（均为预估），同一个问题，不同K值的效果：

K值	平均相似度	答案准确率	预估平均成本	预估响应时间
1	0.92	65%	¥0.07	0.8秒
3	0.85	92%	¥0.22	1.2秒
5	0.78	89%	¥0.36	1.8秒
10	0.65	75%	¥0.86	3.2秒

从表中可以看出，K=3时效果最好（准确率92%），而K=10反而下降到75%，因为引入了噪音。

> [!TIP] 上述测试数据数值，只代表特定问题在特定数据下的测试数据。虽然数值是特例，但其趋势符合预期。这里给出只是为了方便读者理解不是越多越好，也不是越少越好，也并不是每次都是取3条效果最佳。 如果你不追求极致的效果，经验性来说，一般取Top-3。

角度2：相似度阈值过滤

问题场景：用户问： “这款手机防水吗？”

检索结果：

```
Top-1: 相似度 0.85 - "该手机支持IP68级防水"
Top-2: 相似度 0.52 - "该手机支持无线充电"
Top-3: 相似度 0.48 - "该手机电池容量5000mAh"
```

但Top-2和Top-3相似度太低（<0.6），它们和"防水"关系不大，放进上下文只会干扰AI。

解决方案：设置相似度阈值

在检索时设置一个相似度阈值（比如0.7），只保留相似度高于阈值的chunk。具体做法是：先检索10个chunk，然后将向量距离转换为相似度（相似度 = 1 - 距离），过滤掉低于阈值的结果。这样就能只保留Top-1，过滤掉Top-2和Top-3。

效果：成本降低（只传输真正相关的内容）的同时，质量也得到提升（减少噪音，答案更精准）。

角度3：上下文压缩技术

即使筛选后，上下文仍可能很长，有多种压缩技术可用。

技术1：摘要压缩

用大模型把长上下文压缩成摘要：

```
# 伪代码：核心逻辑
def rag_with_summary(question):
    # 检索上下文
    long_context = retrieve_chunks(question)  # 3000字

    # 压缩成摘要
    summary = llm.summarize(long_context, max_length=200)

    # 生成答案
    answer = llm.generate(question, context=summary)

    return answer
```

效果：原始上下文3000字可以压缩到200字，成本节省93%，但摘要会丢失一些细节。这种方法适合背景介绍类内容，以及不需要逐字引用的场景。

技术2：关键句提取

不用摘要，而是提取最相关的几句话：

```
# 伪代码：核心逻辑
def extract_key_sentences(text, question, top_n=3):
    # 分句
    sentences = split_by_period(text)

    # 计算相似度
    scores = calculate_similarity(sentences, question)

    # 排序取Top-N
    top_sentences = get_top_n(scores, n=top_n)

    return join(top_sentences)
```

效果：这种方法保留了原文，但只传输最相关的句子，成本和精准度都能兼顾。

技术3：LlamaIndex框架

如果你用LlamaIndex框架，它内置了上下文压缩功能，可以自动识别并保留最相关的句子，压缩掉不相关的部分。

（具体使用方法我们后续详细讲解）

10.3 避免重复查询：用智慧降低成本

前面讲的是"如何优化上下文质量"（压缩、过滤、提取关键信息），现在我们换个角度：

如果同样的问题被问了100次，每次都重新检索、重新计算、重新调用API，这不是浪费吗？

这一节，我们就来看看如何用"缓存"和"分层"策略，避免这些无谓的重复查询。

分层检索：预先整理高频FAQ

每次都检索整个知识库，即使是简单问题也要扫描百万级向量。

解决思路：把知识库分为两层，预先整理出常见问题：

预先

第一层是高频FAQ（100-200条），提前人工整理最常被问的问题，先在这个小库里用向量检索快速查找。第二层是完整知识库（数万条），包含所有文档，只有FAQ没命中时才来这里检索。

这就像超市把热门商品放在入口，大部分人不用走到里面，直接在门口拿就走。

代码实现：

```
# 伪代码：核心逻辑
class LayeredRAG:
    def __init__(self):
        self.faq_db = load_faq_database()      # FAQ小库（100条）
        self.full_db = load_full_database()    # 完整库（10000条）

    def query(self, question):
        # 先查FAQ
        faq_result = self.faq_db.query(question, n_results=3)

        # 判断第1条相似度
        if get_top1_similarity(faq_result) > 0.7:
            # FAQ命中，说明结果是可取的
            filtered = filter_by_similarity(faq_result, threshold=0.5)
            return generate_answer(question, filtered)
        else:
            # FAQ未命中，查完整库
            full_result = self.full_db.query(question, n_results=3)
            return generate_answer(question, full_result)
```

效果：80%的问题命中FAQ，只需检索100条数据，检索速度提升10倍，成本降低80%。

这里用了两个阈值：

阈值1：0.7（决定是否命中FAQ）

- 只看第1条相似度
- 如果 > 0.7 → 说明FAQ有答案，进入FAQ分支
- 如果 ≤ 0.7 → FAQ没答案，去完整库查

阈值2：0.5（过滤低相似度结果）

- FAQ命中后，取回的3条中，可能只有第1条 > 0.7
- 第2、第3条可能只有0.6、0.3
- 过滤掉 < 0.5 的结果，避免噪音
- 最终传给LLM的1-3条高相似度结果

缓存策略：动态记录已回答的问题

同样的问题被问了100次，每次都重新检索、重新生成。

解决思路：用一个字典动态存储"问题-答案"对：第一次问时正常检索+生成，然后自动存入缓存；第二次问时发现缓存里有，直接返回，不调用API；随着使用，缓存越来越多，命中率越来越高。

和上小节的区别是：上小节的FAQ是预先准备，这里是自动积累的。

上小节的FAQ是预先准备，这里是自动积累的

但缓存匹配不能用简单的字符串对比，因为：

- 用户问：“如何部署Embedding模型？”
- 用户问：“Embedding模型怎么部署？”

意思相同，但文本不同。如果用哈希值匹配，这两个问题会被认为是不同的，无法命中缓存。

解决方案：用向量相似度进行语义匹配：

- 将问题向量化（用Embedding模型）
- 在缓存向量库中查找相似问题
- 如果相似度 > 0.95，认为是同一问题，返回缓存答案
- 否则正常检索+生成，并将新的问题-答案对存入缓存

这样就能识别语义相同但表达不同的问题。

```
# 伪代码：核心逻辑
class CachedRAG:
    def __init__(self):
        self.cache_db = create_vector_db()  # 缓存向量库
        self.rag = RAGSystem()

    def query(self, question):
        # 将问题向量化，查缓存
        cached = self.cache_db.query(question, n_results=1)

        # 判断是否命中
        if get_top1_similarity(cached) > 0.95:  # 高相似度
            # 缓存命中
            return cached['answer']
        else:
            # 未命中，正常RAG
            answer = self.rag.query(question)

            # 存入缓存
            self.cache_db.add(question, answer)

            return answer
```

效果：相同问题第二次查询耗时不到10ms，成本为0（不调用API），缓存命中率通常在30-50%之间（取决于业务场景）。

10.4 下一节预告

到这里，我们已经掌握了从文档处理、向量检索到上下文优化的全流程。**但你可能会发现：**每次都要手写分块、建库、检索的代码，很繁琐。有没有现成的框架可以简化这些流程？

下一节《LlamaIndex实战：RAG开发框架》将带你认识一个强大的RAG框架，看看它如何用几行代码实现我们前面写了上百行才能完成的功能。

10.5 ■ 学点英语

中文	English	音标	说明
问题重写	Query Rewriting	/ˈkwɪri riːˈraɪtɪŋ/	用LLM将用户问题改写得更清晰以提升检索准确率
混合检索	Hybrid Search	/ˈhaɪbrɪd sɜːrtʃ/	组合向量检索、关键词检索、BM25等多种检索方式
重排序模型	Reranking Model	/riːˈræŋkɪŋ ˈmɑːdl/	对初筛结果进行更精确的相关性打分和重排
缓存策略	Cache Strategy	/kæʃ ˈstrætədʒi/	缓存相同或相似问题的答案，避免重复API调用
Token 预算	Token Budget	/ˈtoʊkən ˈbʌdʒɪt/	每次对话允许消耗的最大Token数量